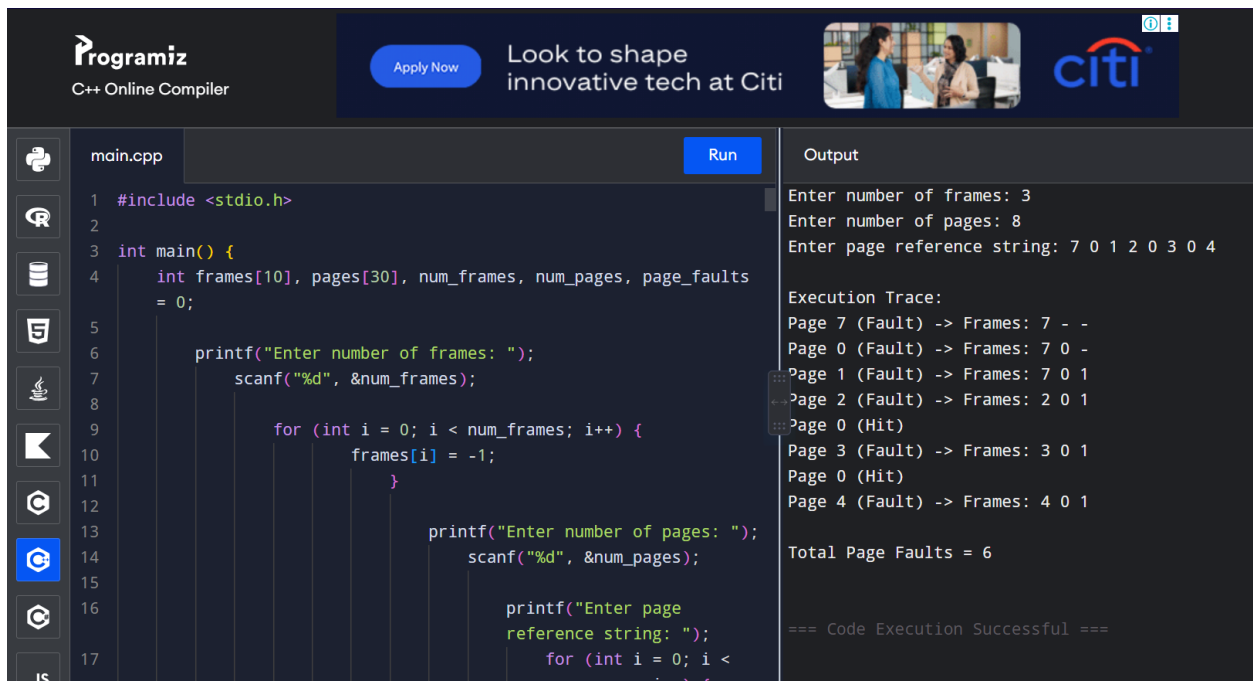


EXPERIMENT 27:

27. Construct a C program to simulate the optimal paging technique of memory management

AIM:

To construct a C program to simulate the Optimal page replacement technique of memory management.



```
main.cpp
1 #include <stdio.h>
2
3 int main() {
4     int frames[10], pages[30], num_frames, num_pages, page_faults = 0;
5
6     printf("Enter number of frames: ");
7     scanf("%d", &num_frames);
8
9     for (int i = 0; i < num_frames; i++) {
10         frames[i] = -1;
11     }
12
13     printf("Enter number of pages: ");
14     scanf("%d", &num_pages);
15
16     printf("Enter page reference string: ");
17     for (int i = 0; i < num_pages; i++) {
```

Output

```
Enter number of frames: 3
Enter number of pages: 8
Enter page reference string: 7 0 1 2 0 3 0 4

Execution Trace:
Page 7 (Fault) -> Frames: 7 - -
Page 0 (Fault) -> Frames: 7 0 -
Page 1 (Fault) -> Frames: 7 0 1
Page 2 (Fault) -> Frames: 2 0 1
Page 0 (Hit)
Page 3 (Fault) -> Frames: 3 0 1
Page 0 (Hit)
Page 4 (Fault) -> Frames: 4 0 1

Total Page Faults = 6

=== Code Execution Successful ===
```

PSEUDO CODE :

BEGIN

PRINT "Enter number of frames:"

READ num_frames

DECLARE frames array of size num_frames

INITIALIZE frames with -1

PRINT "Enter number of pages:"

READ num_pages

```
DECLARE pages array of size num_pages
```

```
PRINT "Enter page reference string:"
```

```
FOR i FROM 0 TO num_pages - 1 DO
```

```
    READ pages[i]
```

```
END FOR
```

```
SET page_faults = 0
```

```
FOR i FROM 0 TO num_pages - 1 DO
```

```
    SET page = pages[i]
```

```
    SET page_found = 0
```

```
    // Check if page exists in frame
```

```
    FOR j FROM 0 TO num_frames - 1 DO
```

```
        IF frames[j] == page THEN
```

```
            SET page_found = 1
```

```
            BREAK
```

```
        END IF
```

```
    END FOR
```

```
IF page_found == 0 THEN
```

```
    SET replace_index = -1
```

```
    SET empty_found = 0
```

```
    // Look for empty frame first
```

```
    FOR j FROM 0 TO num_frames - 1 DO
```

```
        IF frames[j] == -1 THEN
```

```
            SET replace_index = j
```

```
            SET empty_found = 1
```

```

        BREAK
    END IF
END FOR

// Find frame page that will not be used for longest time in future
IF empty_found == 0 THEN
    SET farthest_distance = -1
    FOR j FROM 0 TO num_frames - 1 DO
        SET next_use = INFINITY
        FOR k FROM i + 1 TO num_pages - 1 DO
            IF frames[j] == pages[k] THEN
                SET next_use = k
                BREAK
            END IF
        END FOR
    END FOR

    IF next_use > farthest_distance THEN
        SET farthest_distance = next_use
        SET replace_index = j
    END IF
END FOR

SET frames[replace_index] = page
INCREMENT page_faults
PRINT "Page " + page + " (Fault) -> Frames: " + CURRENT_FRAMES
ELSE
    PRINT "Page " + page + " (Hit)"
END IF
END FOR

```

```
    PRINT "Total Page Faults: " + page_faults
```

```
END
```

CODE:

```
#include <stdio.h>
```

```
int main() {
```

```
    int frames[10],  
    pages[30], num_frames,  
    num_pages, page_faults  
    = 0;
```

```
    printf("Enter number  
of frames: ");
```

```
    scanf("%d",  
&num_frames);
```

```
    for (int i = 0; i <  
num_frames; i++) {
```

```
        frames[i] = -1;
```

```
    }
```

```
    printf("Enter number  
of pages: ");
```

```
    scanf("%d",  
&num_pages);
```

```
    printf("Enter page  
reference string: ");
```

```
    for (int i = 0; i <  
num_pages; i++) {
```

```
        scanf("%d",  
&pages[i]);  
  
    }
```

```
    printf("\nExecution  
Trace:\n");
```

```
    for (int i = 0; i <  
num_pages; i++) {
```

```
        int page_found = 0;
```

```
        for (int j = 0; j <
num_frames; j++) {

            if (frames[j] ==
pages[i]) {

                page_found =
1;

                break;

            }

        }
```

```
        if (page_found ==
0) {

            int replace_index
= -1;

            int empty_found
= 0;
```

```
            for (int j = 0; j <
num_frames; j++) {
```

```

        if (frames[j]
== -1) {

replace_index = j;

empty_found = 1;

        break;

    }

}

if
(!empty_found) {

    int
farthest_use = -1;

    for (int j = 0; j
< num_frames; j++) {

        int next_use
= 9999;

        for (int k = i
+ 1; k < num_pages;

```

```
k++) {  
  
    if  
(frames[j] == pages[k])  
{  
  
    next_use = k;  
  
        break;  
  
    }  
  
    }  
  
    if (next_use  
> farthest_use) {  
  
    farthest_use = next_use;  
  
    replace_index = j;  
  
    }  
  
    }  
  
    }
```

```
frames[replace_index] =  
pages[i];
```

```
    page_faults++;
```

```
    printf("Page %d  
(Fault) -> Frames: ",  
pages[i]);
```

```
        for (int k = 0; k  
< num_frames; k++) {
```

```
            if (frames[k]  
!= -1)
```

```
                printf("%d  
", frames[k]);
```

```
            else
```

```
                printf("- ");
```

```
        }
```

```
        printf("\n");
```

```
    } else {
```

```
        printf("Page %d  
(Hit)\n", pages[i]);  
  
    }  
  
}
```

```
    printf("\nTotal Page  
Faults = %d\n",  
page_faults);  
  
    return 0;  
  
}
```